CHAPTER 14
OVERLOADED OPERATIONS AND CONVERSIONS

CONTENTS

Section 14.1 Basic Concepts 552
Section 14.2 Input and Output Operators 556
Section 14.3 Arithmetic and Relational Operators . . . 560
Section 14.4 Assignment Operators 563
Section 14.5 Subscript Operator 564
Section 14.6 Increment and Decrement Operators . . . 566
Section 14.7 Member Access Operators 569
Section 14.8 Function-Call Operator 571
Section 14.9 Overloading, Conversions, and Operators 579
Chapter Summary 590
Defined Terms 590

In Chapter 4, we saw that C++ defines a large number of operators and automatic conversions among the built-in types. These facilities allow programmers to write a rich set of mixed-type expressions.

C++ lets us define what the operators mean when applied to objects of class type. It also lets us define conversions for class types. Class-type conversions are used like the built-in conversions to implicitly convert an object of one type to another type when needed.

Operator overloading lets us define the meaning of an operator when applied to operand(s) of a class type. Judicious use of operator overloading can make our programs easier to write and easier to read. As an example, because our original `Sales_item` class type (§ 1.5.1, p. 20) defined the input, output, and addition operators, we can print the sum of two `Sales_item`s as

```
cout << item1 + item2;   // print the sum of two Sales_items
```

In contrast, because our `Sales_data` class (§ 7.1, p. 254) does not yet have overloaded operators, code to print their sum is more verbose and, hence, less clear:

```
print(cout, add(data1, data2));   // print the sum of two Sales_datas
```

14.1 Basic Concepts

Overloaded operators are functions with special names: the keyword `operator` followed by the symbol for the operator being defined. Like any other function, an overloaded operator has a return type, a parameter list, and a body.

An overloaded operator function has the same number of parameters as the operator has operands. A unary operator has one parameter; a binary operator has two. In a binary operator, the left-hand operand is passed to the first parameter and the right-hand operand to the second. Except for the overloaded function-call operator, `operator()`, an overloaded operator may not have default arguments (§ 6.5.1, p. 236).

If an operator function is a member function, the first (left-hand) operand is bound to the implicit `this` pointer (§ 7.1.2, p. 257). Because the first operand is implicitly bound to `this`, a member operator function has one less (explicit) parameter than the operator has operands.

When an overloaded operator is a member function, `this` is bound to the left-hand operand. Member operator functions have one less (explicit) parameter than the number of operands.

An operator function must either be a member of a class or have at least one parameter of class type:

```
// error: cannot redefine the built-in operator for ints
int operator+(int, int);
```

This restriction means that we cannot change the meaning of an operator when applied to operands of built-in type.

We can overload most, but not all, of the operators. Table 14.1 shows whether or not an operator may be overloaded. We'll cover overloading `new` and `delete` in § 19.1.1 (p. 820).

We can overload only existing operators and cannot invent new operator symbols. For example, we cannot define `operator**` to provide exponentiation.

Four symbols (`+`, `-`, `*`, and `&`) serve as both unary and binary operators. Either or both of these operators can be overloaded. The number of parameters determines which operator is being defined.

An overloaded operator has the same precedence and associativity (§ 4.1.2, p. 136) as the corresponding built-in operator. Regardless of the operand types

```
x == y + z;
```

is always equivalent to `x == (y + z)`.

Table 14.1: Operators

Operators That May Be Overloaded

`+`	`-`	`*`	`/`	`%`	`^`
`&`	`|`	`~`	`!`	`,`	`=`
`<`	`>`	`<=`	`>=`	`++`	`--`
`<<`	`>>`	`==`	`!=`	`&&`	`||`
`+=`	`-=`	`/=`	`%=`	`^=`	`&=`
`|=`	`*=`	`<<=`	`>>=`	`[]`	`()`
`->`	`->*`	`new`	`new []`	`delete`	`delete []`

Operators That Cannot Be Overloaded

`::`	`.*`	`.`	`?:`

Calling an Overloaded Operator Function Directly

Ordinarily, we "call" an overloaded operator function indirectly by using the operator on arguments of the appropriate type. However, we can also call an overloaded operator function directly in the same way that we call an ordinary function. We name the function and pass an appropriate number of arguments of the appropriate type:

```
// equivalent calls to a nonmember operator function
data1 + data2;              // normal expression
operator+(data1, data2);    // equivalent function call
```

These calls are equivalent: Both call the nonmember function `operator+`, passing `data1` as the first argument and `data2` as the second.

We call a member operator function explicitly in the same way that we call any other member function. We name an object (or pointer) on which to run the function and use the dot (or arrow) operator to fetch the function we wish to call:

```
data1 += data2;             // expression-based ''call''
data1.operator+=(data2);    // equivalent call to a member operator function
```

Each of these statements calls the member function `operator+=`, binding `this` to the address of `data1` and passing `data2` as an argument.

Some Operators Shouldn't Be Overloaded

Recall that a few operators guarantee the order in which operands are evaluated. Because using an overloaded operator is really a function call, these guarantees do not apply to overloaded operators. In particular, the operand-evaluation guarantees of the logical AND, logical OR (§ 4.3, p. 141), and comma (§ 4.10, p. 157)

operators are not preserved. Moreover, overloaded versions of `&&` or `||` operators do not preserve short-circuit evaluation properties of the built-in operators. Both operands are always evaluated.

Because the overloaded versions of these operators do not preserve order of evaluation and/or short-circuit evaluation, it is usually a bad idea to overload them. Users are likely to be surprised when the evaluation guarantees they are accustomed to are not honored for code that happens to use an overloaded version of one of these operators.

Another reason not to overload comma, which also applies to the address-of operator, is that unlike most operators, the language defines what the comma and address-of operators mean when applied to objects of class type. Because these operators have built-in meaning, they ordinarily should not be overloaded. Users of the class will be surprised if these operators behave differently from their normal meanings.

Ordinarily, the comma, address-of, logical AND, and logical OR operators should *not* be overloaded.

Use Definitions That Are Consistent with the Built-in Meaning

When you design a class, you should always think first about what operations the class will provide. Only after you know what operations are needed should you think about whether to define each operation as an ordinary function or as an overloaded operator. Those operations with a logical mapping to an operator are good candidates for defining as overloaded operators:

- If the class does IO, define the shift operators to be consistent with how IO is done for the built-in types.
- If the class has an operation to test for equality, define `operator==`. If the class has `operator==`, it should usually have `operator!=` as well.
- If the class has a single, natural ordering operation, define `operator<`. If the class has `operator<`, it should probably have all of the relational operators.
- The return type of an overloaded operator usually should be compatible with the return from the built-in version of the operator: The logical and relational operators should return `bool`, the arithmetic operators should return a value of the class type, and assignment and compound assignment should return a reference to the left-hand operand.

Assignment and Compound Assignment Operators

Assignment operators should behave analogously to the synthesized operators: After an assignment, the values in the left-hand and right-hand operands should have the same value, and the operator should return a reference to its left-hand operand. Overloaded assignment should generalize the built-in meaning of assignment, not circumvent it.

CAUTION: USE OPERATOR OVERLOADING JUDICIOUSLY

Each operator has an associated meaning from its use on the built-in types. Binary +, for example, is strongly identified with addition. Mapping binary + to an analogous operation for a class type can provide a convenient notational shorthand. For example, the library `string` type, following a convention common to many programming languages, uses + to represent concatenation—"adding" one `string` to the other.

Operator overloading is most useful when there is a logical mapping of a built-in operator to an operation on our type. Using overloaded operators rather than inventing named operations can make our programs more natural and intuitive. Overuse or outright abuse of operator overloading can make our classes incomprehensible.

Obvious abuses of operator overloading rarely happen in practice. As an example, no responsible programmer would define `operator+` to perform subtraction. More common, but still inadvisable, are uses that contort an operator's "normal" meaning to force a fit to a given type. Operators should be used only for operations that are likely to be unambiguous to users. An operator has an ambiguous meaning if it plausibly has more than one interpretation.

If a class has an arithmetic (§ 4.2, p. 139) or bitwise (§ 4.8, p. 152) operator, then it is usually a good idea to provide the corresponding compound-assignment operator as well. Needless to say, the += operator should be defined to behave the same way the built-in operators do: it should behave as + followed by =.

Choosing Member or Nonmember Implementation

When we define an overloaded operator, we must decide whether to make the operator a class member or an ordinary nonmember function. In some cases, there is no choice—some operators are required to be members; in other cases, we may not be able to define the operator appropriately if it is a member.

The following guidelines can be of help in deciding whether to make an operator a member or an ordinary nonmember function:

- The assignment (=), subscript (`[]`), call (`()`), and member access arrow (`->`) operators *must* be defined as members.
- The compound-assignment operators ordinarily *ought* to be members. However, unlike assignment, they are not required to be members.
- Operators that change the state of their object or that are closely tied to their given type—such as increment, decrement, and dereference—usually should be members.
- Symmetric operators—those that might convert either operand, such as the arithmetic, equality, relational, and bitwise operators—usually should be defined as ordinary nonmember functions.

Programmers expect to be able to use symmetric operators in expressions with mixed types. For example, we can add an `int` and a `double`. The addition is symmetric because we can use either type as the left-hand or the right-hand operand.

If we want to provide similar mixed-type expressions involving class objects, then the operator must be defined as a nonmember function.

When we define an operator as a member function, then the left-hand operand must be an object of the class of which that operator is a member. For example:

```
string s = "world";
string t = s + "!";   // ok: we can add a const char* to a string
string u = "hi" + s;  // would be an error if + were a member of string
```

If `operator+` were a member of the `string` class, the first addition would be equivalent to `s.operator+("!")`. Likewise, `"hi" + s` would be equivalent to `"hi".operator+(s)`. However, the type of `"hi"` is `const char*`, and that is a built-in type; it does not even have member functions.

Because `string` defines `+` as an ordinary nonmember function, `"hi" + s` is equivalent to `operator+("hi", s)`. As with any function call, either of the arguments can be converted to the type of the parameter. The only requirements are that at least one of the operands has a class type, and that both operands can be converted (unambiguously) to `string`.

Exercises Section 14.1

Exercise 14.1: In what ways does an overloaded operator differ from a built-in operator? In what ways are overloaded operators the same as the built-in operators?

Exercise 14.2: Write declarations for the overloaded input, output, addition, and compound-assignment operators for `Sales_data`.

Exercise 14.3: Both `string` and `vector` define an overloaded `==` that can be used to compare objects of those types. Assuming `svec1` and `svec2` are `vectors` that hold `strings`, identify which version of `==` is applied in each of the following expressions:

(a) `"cobble" == "stone"` (b) `svec1[0] == svec2[0]`
(c) `svec1 == svec2` (d) `"svec1[0] == "stone"`

Exercise 14.4: Explain how to decide whether the following should be class members:

(a) `%` (b) `%=` (c) `++` (d) `->` (e) `<<` (f) `&&` (g) `==` (h) `()`

Exercise 14.5: In exercise 7.40 from § 7.5.1 (p. 291) you wrote a sketch of one of the following classes. Decide what, if any, overloaded operators your class should provide.

(a) `Book` (b) `Date` (c) `Employee`
(d) `Vehicle` (e) `Object` (f) `Tree`

14.2 Input and Output Operators

As we've seen, the IO library uses `>>` and `<<` for input and output, respectively. The IO library itself defines versions of these operators to read and write the built-in types. Classes that support IO ordinarily define versions of these operators for objects of the class type.

14.2.1 Overloading the Output Operator <<

Ordinarily, the first parameter of an output operator is a reference to a nonconst `ostream` object. The `ostream` is nonconst because writing to the stream changes its state. The parameter is a reference because we cannot copy an `ostream` object.

The second parameter ordinarily should be a reference to `const` of the class type we want to print. The parameter is a reference to avoid copying the argument. It can be `const` because (ordinarily) printing an object does not change that object.

To be consistent with other output operators, `operator<<` normally returns its `ostream` parameter.

The `Sales_data` Output Operator

As an example, we'll write the `Sales_data` output operator:

```
ostream &operator<<(ostream &os, const Sales_data &item)
{
    os << item.isbn() << " " << item.units_sold << " "
       << item.revenue << " " << item.avg_price();
    return os;
}
```

Except for its name, this function is identical to our earlier `print` function (§ 7.1.3, p. 261). Printing a `Sales_data` entails printing its three data elements and the computed average sales price. Each element is separated by a space. After printing the values, the operator returns a reference to the `ostream` it just wrote.

Output Operators Usually Do Minimal Formatting

The output operators for the built-in types do little if any formatting. In particular, they do not print newlines. Users expect class output operators to behave similarly. If the operator does print a newline, then users would be unable to print descriptive text along with the object on the same line. An output operator that does minimal formatting lets users control the details of their output.

Generally, output operators should print the contents of the object, with minimal formatting. They should not print a newline.

IO Operators Must Be Nonmember Functions

Input and output operators that conform to the conventions of the `iostream` library must be ordinary nonmember functions. These operators cannot be members of our own class. If they were, then the left-hand operand would have to be an object of our class type:

```
Sales_data data;
data << cout; // if operator<< is a member of Sales_data
```

If these operators are members of any class, they would have to be members of `istream` or `ostream`. However, those classes are part of the standard library, and we cannot add members to a class in the library.

Thus, if we want to define the IO operators for our types, we must define them as nonmember functions. Of course, IO operators usually need to read or write the nonpublic data members. As a consequence, IO operators usually must be declared as friends (§ 7.2.1, p. 269).

Exercises Section 14.2.1

Exercise 14.6: Define an output operator for your `Sales_data` class.

Exercise 14.7: Define an output operator for you `String` class you wrote for the exercises in § 13.5 (p. 531).

Exercise 14.8: Define an output operator for the class you chose in exercise 7.40 from § 7.5.1 (p. 291).

14.2.2 Overloading the Input Operator >>

Ordinarily the first parameter of an input operator is a reference to the stream from which it is to read, and the second parameter is a reference to the (nonconst) object into which to read. The operator usually returns a reference to its given stream. The second parameter must be nonconst because the purpose of an input operator is to read data into this object.

The `Sales_data` Input Operator

As an example, we'll write the `Sales_data` input operator:

```
istream &operator>>(istream &is, Sales_data &item)
{
    double price;  // no need to initialize; we'll read into price before we use it
    is >> item.bookNo >> item.units_sold >> price;
    if (is)        // check that the inputs succeeded
        item.revenue = item.units_sold * price;
    else
        item = Sales_data(); // input failed: give the object the default state
    return is;
}
```

Except for the `if` statement, this definition is similar to our earlier `read` function (§ 7.1.3, p. 261). The `if` checks whether the reads were successful. If an IO error occurs, the operator resets its given object to the empty `Sales_data`. That way, the object is guaranteed to be in a consistent state.

Input operators must deal with the possibility that the input might fail; output operators generally don't bother.

Errors during Input

The kinds of errors that might happen in an input operator include the following:

- A read operation might fail because the stream contains data of an incorrect type. For example, after reading `bookNo`, the input operator assumes that the next two items will be numeric data. If nonnumeric data is input, that read and any subsequent use of the stream will fail.
- Any of the reads could hit end-of-file or some other error on the input stream.

Rather than checking each read, we check once after reading all the data and before using those data:

```
if (is)          //  check that the inputs succeeded
    item.revenue = item.units_sold * price;
else
    item = Sales_data(); //  input failed: give the object the default state
```

If any of the read operations fails, `price` will have an undefined value. Therefore, before using `price`, we check that the input stream is still valid. If it is, we do the calculation and store the result in `revenue`. If there was an error, we do not worry about which input failed. Instead, we reset the entire object to the empty `Sales_data` by assigning a new, default-initialized `Sales_data` object to `item`. After this assignment, `item` will have an empty `string` for its `bookNo` member, and its `revenue` and `units_sold` members will be zero.

Putting the object into a valid state is especially important if the object might have been partially changed before the error occurred. For example, in this input operator, we might encounter an error after successfully reading a new `bookNo`. An error after reading `bookNo` would mean that the `units_sold` and `revenue` members of the old object were unchanged. The effect would be to associate a different `bookNo` with those data.

By leaving the object in a valid state, we (somewhat) protect a user that ignores the possibility of an input error. The object will be in a usable state—its members are all defined. Similarly, the object won't generate misleading results—its data are internally consistent.

Input operators should decide what, if anything, to do about error recovery.

Indicating Errors

Some input operators need to do additional data verification. For example, our input operator might check that the `bookNo` we read is in an appropriate format. In such cases, the input operator might need to set the stream's condition state to indicate failure (§ 8.1.2, p. 312), even though technically speaking the actual IO was successful. Usually an input operator should set only the `failbit`. Setting `eofbit` would imply that the file was exhausted, and setting `badbit` would indicate that the stream was corrupted. These errors are best left to the IO library itself to indicate.

EXERCISES SECTION 14.2.2

Exercise 14.9: Define an input operator for your `Sales_data` class.

Exercise 14.10: Describe the behavior of the `Sales_data` input operator if given the following input:

(a) `0-201-99999-9 10 24.95` (b) `10 24.95 0-210-99999-9`

Exercise 14.11: What, if anything, is wrong with the following `Sales_data` input operator? What would happen if we gave this operator the data in the previous exercise?

```
istream& operator>>(istream& in, Sales_data& s)
{
    double price;
    in >> s.bookNo >> s.units_sold >> price;
    s.revenue = s.units_sold * price;
    return in;
}
```

Exercise 14.12: Define an input operator for the class you used in exercise 7.40 from § 7.5.1 (p. 291). Be sure the operator handles input errors.

14.3 Arithmetic and Relational Operators

Ordinarily, we define the arithmetic and relational operators as nonmember functions in order to allow conversions for either the left- or right-hand operand (§ 14.1, p. 555). These operators shouldn't need to change the state of either operand, so the parameters are ordinarily references to `const`.

An arithmetic operator usually generates a new value that is the result of a computation on its two operands. That value is distinct from either operand and is calculated in a local variable. The operation returns a copy of this local as its result. Classes that define an arithmetic operator generally define the corresponding compound assignment operator as well. When a class has both operators, it is usually more efficient to define the arithmetic operator to use compound assignment:

```
// assumes that both objects refer to the same book
Sales_data
operator+(const Sales_data &lhs, const Sales_data &rhs)
{
    Sales_data sum = lhs;  // copy data members from lhs into sum
    sum += rhs;            // add rhs into sum
    return sum;
}
```

This definition is essentially identical to our original `add` function (§ 7.1.3, p. 261). We copy `lhs` into the local variable `sum`. We then use the `Sales_data` compound-assignment operator (which we'll define on page 564) to add the values from `rhs` into `sum`. We end the function by returning a copy of `sum`.

Classes that define both an arithmetic operator and the related compound assignment ordinarily ought to implement the arithmetic operator by using the compound assignment.

EXERCISES SECTION 14.3

Exercise 14.13: Which other arithmetic operators (Table 4.1 (p. 139)), if any, do you think `Sales_data` ought to support? Define any you think the class should include.

Exercise 14.14: Why do you think it is more efficient to define `operator+` to call `operator+=` rather than the other way around?

Exercise 14.15: Should the class you chose for exercise 7.40 from § 7.5.1 (p. 291) define any of the arithmetic operators? If so, implement them. If not, explain why not.

14.3.1 Equality Operators

Ordinarily, classes in C++ define the equality operator to test whether two objects are equivalent. That is, they usually compare every data member and treat two objects as equal if and only if all the corresponding members are equal. In line with this design philosophy, our `Sales_data` equality operator should compare the `bookNo` as well as the sales figures:

```
bool operator==(const Sales_data &lhs, const Sales_data &rhs)
{
    return lhs.isbn() == rhs.isbn() &&
           lhs.units_sold == rhs.units_sold &&
           lhs.revenue == rhs.revenue;
}
bool operator!=(const Sales_data &lhs, const Sales_data &rhs)
{
    return !(lhs == rhs);
}
```

The definition of these functions is trivial. More important are the design principles that these functions embody:

- If a class has an operation to determine whether two objects are equal, it should define that function as `operator==` rather than as a named function: Users will expect to be able to compare objects using ==; providing == means they won't need to learn and remember a new name for the operation; and it is easier to use the library containers and algorithms with classes that define the == operator.
- If a class defines `operator==`, that operator ordinarily should determine whether the given objects contain equivalent data.

- Ordinarily, the equality operator should be transitive, meaning that if `a == b` and `b == c` are both true, then `a == c` should also be true.
- If a class defines `operator==`, it should also define `operator!=`. Users will expect that if they can use `==` then they can also use `!=`, and vice versa.
- One of the equality or inequality operators should delegate the work to the other. That is, one of these operators should do the real work to compare objects. The other should call the one that does the real work.

Classes for which there is a logical meaning for equality normally should define `operator==`. Classes that define `==` make it easier for users to use the class with the library algorithms.

Exercises Section 14.3.1

Exercise 14.16: Define equality and inequality operators for your `StrBlob` (§ 12.1.1, p. 456), `StrBlobPtr` (§ 12.1.6, p. 474), `StrVec` (§ 13.5, p. 526), and `String` (§ 13.5, p. 531) classes.

Exercise 14.17: Should the class you chose for exercise 7.40 from § 7.5.1 (p. 291) define the equality operators? If so, implement them. If not, explain why not.

14.3.2 Relational Operators

Classes for which the equality operator is defined also often (but not always) have relational operators. In particular, because the associative containers and some of the algorithms use the less-than operator, it can be useful to define an `operator<`.

Ordinarily the relational operators should

1. Define an ordering relation that is consistent with the requirements for use as a key to an associative container (§ 11.2.2, p. 424); and
2. Define a relation that is consistent with `==` if the class has both operators. In particular, if two objects are `!=`, then one object should be `<` the other.

Although we might think our `Sales_data` class should support the relational operators, it turns out that it probably should not do so. The reasons are subtle and are worth understanding.

We might think that we'd define `<` similarly to `compareIsbn` (§ 11.2.2, p. 425). That function compared `Sales_data` objects by comparing their ISBNs. Although `compareIsbn` provides an ordering relation that meets requirment 1, that function yields results that are inconsistent with our definition of `==`. As a result, it does not meet requirement 2.

The `Sales_data` `==` operator treats two transactions with the same ISBN as unequal if they have different `revenue` or `units_sold` members. If we defined

the < operator to compare only the ISBN member, then two objects with the same ISBN but different `units_sold` or `revenue` would compare as unequal, but neither object would be less than the other. Ordinarily, if we have two objects, neither of which is less than the other, then we expect that those objects are equal.

We might think that we should, therefore, define `operator<` to compare each data element in turn. We could define `operator<` to compare objects with equal `isbn`s by looking next at the `units_sold` and then at the `revenue` members.

However, there is nothing essential about this ordering. Depending on how we plan to use the class, we might want to define the order based first on either `revenue` or `units_sold`. We might want those objects with fewer `units_sold` to be "less than" those with more. Or we might want to consider those with smaller `revenue` "less than" those with more.

For `Sales_data`, there is no single logical definition of <. Thus, it is better for this class not to define < at all.

If a single logical definition for < exists, classes usually should define the < operator. However, if the class also has ==, define < only if the definitions of < and == yield consistent results.

Exercises Section 14.3.2

Exercise 14.18: Define relational operators for your `StrBlob`, `StrBlobPtr`, `StrVec`, and `String` classes.

Exercise 14.19: Should the class you chose for exercise 7.40 from § 7.5.1 (p. 291) define the relational operators? If so, implement them. If not, explain why not.

14.4 Assignment Operators

In addition to the copy- and move-assignment operators that assign one object of the class type to another object of the same type (§ 13.1.2, p. 500, and § 13.6.2, p. 536), a class can define additional assignment operators that allow other types as the right-hand operand.

As one example, in addition to the copy- and move-assignment operators, the library `vector` class defines a third assignment operator that takes a braced list of elements (§ 9.2.5, p. 337). We can use this operator as follows:

```
vector<string> v;
v = {"a", "an", "the"};
```

We can add this operator to our `StrVec` class (§ 13.5, p. 526) as well:

```
class StrVec {
public:
    StrVec &operator=(std::initializer_list<std::string>);
    // other members as in § 13.5 (p. 526)
};
```

To be consistent with assignment for the built-in types (and with the copy- and move-assignment operators we already defined), our new assignment operator will return a reference to its left-hand operand:

```
StrVec &StrVec::operator=(initializer_list<string> il)
{
    // alloc_n_copy allocates space and copies elements from the given range
    auto data = alloc_n_copy(il.begin(), il.end());
    free();    // destroy the elements in this object and free the space
    elements = data.first; // update data members to point to the new space
    first_free = cap = data.second;
    return *this;
}
```

As with the copy- and move-assignment operators, other overloaded assignment operators have to free the existing elements and create new ones. Unlike the copy- and move-assignment operators, this operator does not need to check for self-assignment. The parameter is an `initializer_list<string>` (§ 6.2.6, p. 220), which means that `il` cannot be the same object as the one denoted by `this`.

Assignment operators can be overloaded. Assignment operators, regardless of parameter type, must be defined as member functions.

Compound-Assignment Operators

Compound assignment operators are not required to be members. However, we prefer to define all assignments, including compound assignments, in the class. For consistency with the built-in compound assignment, these operators should return a reference to their left-hand operand. For example, here is the definition of the `Sales_data` compound-assignment operator:

```
// member binary operator: left-hand operand is bound to the implicit this pointer
// assumes that both objects refer to the same book
Sales_data& Sales_data::operator+=(const Sales_data &rhs)
{
    units_sold += rhs.units_sold;
    revenue += rhs.revenue;
    return *this;
}
```

Assignment operators must, and ordinarily compound-assignment operators should, be defined as members. These operators should return a reference to the left-hand operand.

14.5 Subscript Operator

Classes that represent containers from which elements can be retrieved by position often define the subscript operator, `operator[]`.

Exercises Section 14.4

Exercise 14.20: Define the addition and compound-assignment operators for your `Sales_data` class.

Exercise 14.21: Write the `Sales_data` operators so that `+` does the actual addition and `+=` calls `+`. Discuss the disadvantages of this approach compared to the way these operators were defined in § 14.3 (p. 560) and § 14.4 (p. 564).

Exercise 14.22: Define a version of the assignment operator that can assign a `string` representing an ISBN to a `Sales_data`.

Exercise 14.23: Define an `initializer_list` assignment operator for your version of the `StrVec` class.

Exercise 14.24: Decide whether the class you used in exercise 7.40 from § 7.5.1 (p. 291) needs a copy- and move-assignment operator. If so, define those operators.

Exercise 14.25: Implement any other assignment operators your class should define. Explain which types should be used as operands and why.

The subscript operator must be a member function.

To be compatible with the ordinary meaning of subscript, the subscript operator usually returns a reference to the element that is fetched. By returning a reference, subscript can be used on either side of an assignment. Consequently, it is also usually a good idea to define both `const` and nonconst versions of this operator. When applied to a `const` object, subscript should return a reference to `const` so that it is not possible to assign to the returned object.

If a class has a subscript operator, it usually should define two versions: one that returns a plain reference and the other that is a `const` member and returns a reference to `const`.

As an example, we'll define subscript for `StrVec` (§ 13.5, p. 526):

```
class StrVec {
public:
    std::string& operator[](std::size_t n)
        { return elements[n]; }
    const std::string& operator[](std::size_t n) const
        { return elements[n]; }
    //  other members as in § 13.5 (p. 526)
private:
    std::string *elements;    //  pointer to the first element in the array
};
```

We can use these operators similarly to how we subscript a `vector` or array. Because subscript returns a reference to an element, if the `StrVec` is nonconst, we can assign to that element; if we subscript a `const` object, we can't:

```
// assume svec is a StrVec
const StrVec cvec = svec; // copy elements from svec into cvec
// if svec has any elements, run the string empty function on the first one
if (svec.size() && svec[0].empty())  {
    svec[0] = "zero"; // ok: subscript returns a reference to a string
    cvec[0] = "Zip";  // error: subscripting cvec returns a reference to const
}
```

Exercises Section 14.5

Exercise 14.26: Define subscript operators for your `StrVec`, `String`, `StrBlob`, and `StrBlobPtr` classes.

14.6 Increment and Decrement Operators

The increment (++) and decrement (--) operators are most often implemented for iterator classes. These operators let the class move between the elements of a sequence. There is no language requirement that these operators be members of the class. However, because these operators change the state of the object on which they operate, our preference is to make them members.

For the built-in types, there are both prefix and postfix versions of the increment and decrement operators. Not surprisingly, we can define both the prefix and postfix instances of these operators for our own classes as well. We'll look at the prefix versions first and then implement the postfix ones.

Classes that define increment or decrement operators should define both the prefix and postfix versions. These operators usually should be defined as members.

Defining Prefix Increment/Decrement Operators

To illustrate the increment and decrement operators, we'll define these operators for our `StrBlobPtr` class (§ 12.1.6, p. 474):

```
class StrBlobPtr {
public:
    // increment and decrement
    StrBlobPtr& operator++();          // prefix operators
    StrBlobPtr& operator--();
    // other members as before
};
```

To be consistent with the built-in operators, the prefix operators should return a reference to the incremented or decremented object.

The increment and decrement operators work similarly to each other—they call `check` to verify that the `StrBlobPtr` is still valid. If so, `check` also verifies that its given index is valid. If `check` doesn't throw an exception, these operators return a reference to this object.

In the case of increment, we pass the current value of `curr` to `check`. So long as that value is less than the size of the underlying `vector`, `check` will return. If `curr` is already at the end of the `vector`, `check` will throw:

```
// prefix: return a reference to the incremented/decremented object
StrBlobPtr& StrBlobPtr::operator++()
{
    // if curr already points past the end of the container, can't increment it
    check(curr, "increment past end of StrBlobPtr");
    ++curr;       // advance the current state
    return *this;
}

StrBlobPtr& StrBlobPtr::operator--()
{
    // if curr is zero, decrementing it will yield an invalid subscript
    --curr;       // move the current state back one element
    check(curr, "decrement past begin of StrBlobPtr");
    return *this;
}
```

The decrement operator decrements `curr` before calling `check`. That way, if `curr` (which is an `unsigned` number) is already zero, the value that we pass to `check` will be a large positive value representing an invalid subscript (§ 2.1.2, p. 36).

Differentiating Prefix and Postfix Operators

There is one problem with defining both the prefix and postfix operators: Normal overloading cannot distinguish between these operators. The prefix and postfix versions use the same symbol, meaning that the overloaded versions of these operators have the same name. They also have the same number and type of operands.

To solve this problem, the postfix versions take an extra (unused) parameter of type `int`. When we use a postfix operator, the compiler supplies 0 as the argument for this parameter. Although the postfix function can use this extra parameter, it usually should not. That parameter is not needed for the work normally performed by a postfix operator. Its sole purpose is to distinguish a postfix function from the prefix version.

We can now add the postfix operators to `StrBlobPtr`:

```
class StrBlobPtr {
public:
    // increment and decrement
    StrBlobPtr operator++(int);      // postfix operators
    StrBlobPtr operator--(int);
    // other members as before
};
```

To be consistent with the built-in operators, the postfix operators should return the old (unincremented or undecremented) value. That value is returned as a value, not a reference.

The postfix versions have to remember the current state of the object before incrementing the object:

```
// postfix: increment/decrement the object but return the unchanged value
StrBlobPtr StrBlobPtr::operator++(int)
{
    // no check needed here; the call to prefix increment will do the check
    StrBlobPtr ret = *this;    // save the current value
    ++*this;       // advance one element; prefix ++ checks the increment
    return ret;    // return the saved state
}
StrBlobPtr StrBlobPtr::operator--(int)
{
    // no check needed here; the call to prefix decrement will do the check
    StrBlobPtr ret = *this;    // save the current value
    --*this;       // move backward one element; prefix -- checks the decrement
    return ret;    // return the saved state
}
```

Each of our operators calls its own prefix version to do the actual work. For example, the postfix increment operator executes

```
++*this
```

This expression calls the prefix increment operator. That operator checks that the increment is safe and either throws an exception or increments `curr`. Assuming `check` doesn't throw an exception, the postfix functions return the stored copy in `ret`. Thus, after the return, the object itself has been advanced, but the value returned reflects the original, unincremented value.

The `int` parameter is not used, so we do not give it a name.

Calling the Postfix Operators Explicitly

As we saw on page 553, we can explicitly call an overloaded operator as an alternative to using it as an operator in an expression. If we want to call the postfix version using a function call, then we must pass a value for the integer argument:

```
StrBlobPtr p(a1); // p points to the vector inside a1
p.operator++(0);  // call postfix operator++
p.operator++();   // call prefix operator++
```

The value passed usually is ignored but is necessary in order to tell the compiler to use the postfix version.

Exercises Section 14.6

Exercise 14.27: Add increment and decrement operators to your `StrBlobPtr` class.

Exercise 14.28: Define addition and subtraction for `StrBlobPtr` so that these operators implement pointer arithmetic (§ 3.5.3, p. 119).

Exercise 14.29: We did not define a `const` version of the increment and decrement operators. Why not?

14.7 Member Access Operators

The dereference (`*`) and arrow (`->`) operators are often used in classes that represent iterators and in smart pointer classes (§ 12.1, p. 450). We can logically add these operators to our `StrBlobPtr` class as well:

```
class StrBlobPtr {
public:
    std::string& operator*() const
    { auto p = check(curr, "dereference past end");
      return (*p)[curr];  //  (*p) is the vector to which this object points
    }
    std::string* operator->() const
    { //  delegate the real work to the dereference operator
     return & this->operator*();
    }
    //  other members as before
};
```

The dereference operator checks that `curr` is still in range and, if so, returns a reference to the element denoted by `curr`. The arrow operator avoids doing any work of its own by calling the dereference operator and returning the address of the element returned by that operator.

Operator arrow must be a member. The dereference operator is not required to be a member but usually should be a member as well.

It is worth noting that we've defined these operators as `const` members. Unlike the increment and decrment operators, fetching an element doesn't change the state of a `StrBlobPtr`. Also note that these operators return a reference or pointer to nonconst `string`. They do so because we know that a `StrBlobPtr` can only be bound to a nonconst `StrBlob` (§ 12.1.6, p. 474).

We can use these operators the same way that we've used the corresponding operations on pointers or `vector` iterators:

```
StrBlob a1 = {"hi", "bye", "now"};
StrBlobPtr p(a1);             //  p points to the vector inside a1
*p = "okay";                  //  assigns to the first element in a1
cout << p->size() << endl;    //  prints 4, the size of the first element in a1
cout << (*p).size() << endl; //  equivalent to p->size()
```

Constraints on the Return from Operator Arrow

As with most of the other operators (although it would be a bad idea to do so), we can define `operator*` to do whatever processing we like. That is, we can define `operator*` to return a fixed value, say, `42`, or print the contents of the object to which it is applied, or whatever. The same is not true for overloaded arrow. The arrow operator never loses its fundamental meaning of member access. When we overload arrow, we change the object from which arrow fetches the specified member. We cannot change the fact that arrow fetches a member.

When we write `point->mem`, `point` must be a pointer to a class object or it must be an object of a class with an overloaded `operator->`. Depending on the type of `point`, writing `point->mem` is equivalent to

```
(*point).mem;             // point is a built-in pointer type
point.operator()->mem;    // point is an object of class type
```

Otherwise the code is in error. That is, `point->mem` executes as follows:

1. If `point` is a pointer, then the built-in arrow operator is applied, which means this expression is a synonym for `(*point).mem`. The pointer is dereferenced and the indicated member is fetched from the resulting object. If the type pointed to by `point` does not have a member named `mem`, then the code is in error.

2. If `point` is an object of a class that defines `operator->`, then the result of `point.operator->()` is used to fetch `mem`. If that result is a pointer, then step 1 is executed on that pointer. If the result is an object that itself has an overloaded `operator->()`, then this step is repeated on that object. This process continues until either a pointer to an object with the indicated member is returned or some other value is returned, in which case the code is in error.

The overloaded arrow operator *must* return either a pointer to a class type or an object of a class type that defines its own operator arrow.

Exercises Section 14.7

Exercise 14.30: Add dereference and arrow operators to your `StrBlobPtr` class and to the `ConstStrBlobPtr` class that you defined in exercise 12.22 from § 12.1.6 (p. 476). Note that the operators in `constStrBlobPtr` must return `const` references because the `data` member in `constStrBlobPtr` points to a `const vector`.

Exercise 14.31: Our `StrBlobPtr` class does not define the copy constructor, assignment operator, or a destructor. Why is that okay?

Exercise 14.32: Define a class that holds a pointer to a `StrBlobPtr`. Define the overloaded arrow operator for that class.

14.8 Function-Call Operator

Classes that overload the call operator allow objects of its type to be used as if they were a function. Because such classes can also store state, they can be more flexible than ordinary functions.

As a simple example, the following `struct`, named `absInt`, has a call operator that returns the absolute value of its argument:

```
struct absInt {
    int operator()(int val) const {
        return val < 0 ? -val : val;
    }
};
```

This class defines a single operation: the function-call operator. That operator takes an argument of type `int` and returns the argument's absolute value.

We use the call operator by applying an argument list to an `absInt` object in a way that looks like a function call:

```
int i = -42;
absInt absObj;       // object that has a function-call operator
int ui = absObj(i); // passes i to absObj.operator()
```

Even though `absObj` is an object, not a function, we can "call" this object. Calling an object runs its overloaded call operator. In this case, that operator takes an `int` value and returns its absolute value.

The function-call operator must be a member function. A class may define multiple versions of the call operator, each of which must differ as to the number or types of their parameters.

Objects of classes that define the call operator are referred to as **function objects**. Such objects "act like functions" because we can call them.

Function-Object Classes with State

Like any other class, a function-object class can have additional members aside from `operator()`. Function-object classes often contain data members that are used to customize the operations in the call operator.

As an example, we'll define a class that prints a `string` argument. By default, our class will write to `cout` and will print a space following each `string`. We'll also let users of our class provide a different stream on which to write and provide a different separator. We can define this class as follows:

```
class PrintString {
public:
    PrintString(ostream &o = cout, char c = ' '):
        os(o), sep(c) { }
    void operator()(const string &s) const { os << s << sep; }
private:
    ostream &os;    // stream on which to write
    char sep;       // character to print after each output
};
```

Our class has a constructor that takes a reference to an output stream and a character to use as the separator. It uses `cout` and a space as default arguments (§ 6.5.1, p. 236) for these parameters. The body of the function-call operator uses these members when it prints the given `string`.

When we define `PrintString` objects, we can use the defaults or supply our own values for the separator or output stream:

```
PrintString printer;    // uses the defaults; prints to cout
printer(s);             // prints s followed by a space on cout
PrintString errors(cerr, '\n');
errors(s);              // prints s followed by a newline on cerr
```

Function objects are most often used as arguments to the generic algorithms. For example, we can use the library `for_each` algorithm (§ 10.3.2, p. 391) and our `PrintString` class to print the contents of a container:

```
for_each(vs.begin(), vs.end(), PrintString(cerr, '\n'));
```

The third argument to `for_each` is a temporary object of type `PrintString` that we initialize from `cerr` and a newline character. The call to `for_each` will print each element in `vs` to `cerr` followed by a newline.

Exercises Section 14.8

Exercise 14.33: How many operands may an overloaded function-call operator take?

Exercise 14.34: Define a function-object class to perform an if-then-else operation: The call operator for this class should take three parameters. It should test its first parameter and if that test succeeds, it should return its second parameter; otherwise, it should return its third parameter.

Exercise 14.35: Write a class like `PrintString` that reads a line of input from an `istream` and returns a `string` representing what was read. If the read fails, return the empty `string`.

Exercise 14.36: Use the class from the previous exercise to read the standard input, storing each line as an element in a `vector`.

Exercise 14.37: Write a class that tests whether two values are equal. Use that object and the library algorithms to write a program to replace all instances of a given value in a sequence.

14.8.1 Lambdas Are Function Objects

In the previous section, we used a `PrintString` object as an argument in a call to `for_each`. This usage is similar to the programs we wrote in § 10.3.2 (p. 388) that used lambda expressions. When we write a lambda, the compiler translates that expression into an unnamed object of an unnamed class (§ 10.3.3, p. 392). The

classes generated from a lambda contain an overloaded function-call operator. For example, the lambda that we passed as the last argument to `stable_sort`:

```
// sort words by size, but maintain alphabetical order for words of the same size
stable_sort(words.begin(), words.end(),
            [](const string &a, const string &b)
              { return a.size() < b.size();});
```

acts like an unnamed object of a class that would look something like

```
class ShorterString {
public:
    bool operator()(const string &s1, const string &s2) const
    { return s1.size() < s2.size(); }
};
```

The generated class has a single member, which is a function-call operator that takes two `string`s and compares their lengths. The parameter list and function body are the same as the lambda. As we saw in § 10.3.3 (p. 395), by default, lambdas may not change their captured variables. As a result, by default, the function-call operator in a class generated from a lambda is a `const` member function. If the lambda is declared as `mutable`, then the call operator is not `const`.

We can rewrite the call to `stable_sort` to use this class instead of the lambda expression:

```
stable_sort(words.begin(), words.end(), ShorterString());
```

The third argument is a newly constructed `ShorterString` object. The code in `stable_sort` will "call" this object each time it compares two `string`s. When the object is called, it will execute the body of its call operator, returning `true` if the first `string`'s size is less than the second's.

Classes Representing Lambdas with Captures

As we've seen, when a lambda captures a variable by reference, it is up to the program to ensure that the variable to which the reference refers exists when the lambda is executed (§ 10.3.3, p. 393). Therefore, the compiler is permitted to use the reference directly without storing that reference as a data member in the generated class.

In contrast, variables that are captured by value are copied into the lambda (§ 10.3.3, p. 392). As a result, classes generated from lambdas that capture variables by value have data members corresponding to each such variable. These classes also have a constructor to initialize these data members from the value of the captured variables. As an example, in § 10.3.2 (p. 390), the lambda that we used to find the first `string` whose length was greater than or equal to a given bound:

```
// get an iterator to the first element whose size() is >= sz
auto wc = find_if(words.begin(), words.end(),
            [sz](const string &a)
```

would generate a class that looks something like

```
class SizeComp {
    SizeComp(size_t n): sz(n) { }  // parameter for each captured variable
    // call operator with the same return type, parameters, and body as the lambda
    bool operator()(const string &s) const
        { return s.size() >= sz; }
private:
    size_t sz;  // a data member for each variable captured by value
};
```

Unlike our `ShorterString` class, this class has a data member and a constructor to initialize that member. This synthesized class does not have a default constructor; to use this class, we must pass an argument:

```
// get an iterator to the first element whose size() is >= sz
auto wc = find_if(words.begin(), words.end(), SizeComp(sz));
```

Classes generated from a lambda expression have a deleted default constructor, deleted assignment operators, and a default destructor. Whether the class has a defaulted or deleted copy/move constructor depends in the usual ways on the types of the captured data members (§ 13.1.6, p. 508, and § 13.6.2, p. 537).

EXERCISES SECTION 14.8.1

Exercise 14.38: Write a class that tests whether the length of a given `string` matches a given bound. Use that object to write a program to report how many words in an input file are of sizes 1 through 10 inclusive.

Exercise 14.39: Revise the previous program to report the count of words that are sizes 1 through 9 and 10 or more.

Exercise 14.40: Rewrite the `biggies` function from § 10.3.2 (p. 391) to use function-object classes in place of lambdas.

Exercise 14.41: Why do you suppose the new standard added lambdas? Explain when you would use a lambda and when you would write a class instead.

14.8.2 Library-Defined Function Objects

The standard library defines a set of classes that represent the arithmetic, relational, and logical operators. Each class defines a call operator that applies the named operation. For example, the `plus` class has a function-call operator that applies `+` to a pair of operands; the `modulus` class defines a call operator that applies the binary `%` operator; the `equal_to` class applies `==`; and so on.

These classes are templates to which we supply a single type. That type specifies the parameter type for the call operator. For example, `plus<string>` applies the `string` addition operator to `string` objects; for `plus<int>` the operands are `int`s; `plus<Sales_data>` applies `+` to `Sales_data`s; and so on:

```
plus<int> intAdd;         // function object that can add two int values
negate<int>  intNegate;   // function object that can negate an int value
// uses intAdd::operator(int, int) to add 10 and 20
int sum = intAdd(10, 20);          // equivalent to sum = 30
sum = intNegate(intAdd(10, 20));   // equivalent to sum = -30
// uses intNegate::operator(int) to generate -10 as the second parameter
// to intAdd::operator(int, int)
sum = intAdd(10, intNegate(10));   // sum = 0
```

These types, listed in Table 14.2, are defined in the `functional` header.

Table 14.2: Library Function Objects

Arithmetic	*Relational*	*Logical*
`plus<Type>`	`equal_to<Type>`	`logical_and<Type>`
`minus<Type>`	`not_equal_to<Type>`	`logical_or<Type>`
`multiplies<Type>`	`greater<Type>`	`logical_not<Type>`
`divides<Type>`	`greater_equal<Type>`	
`modulus<Type>`	`less<Type>`	
`negate<Type>`	`less_equal<Type>`	

Using a Library Function Object with the Algorithms

The function-object classes that represent operators are often used to override the default operator used by an algorithm. As we've seen, by default, the sorting algorithms use `operator<`, which ordinarily sorts the sequence into ascending order. To sort into descending order, we can pass an object of type `greater`. That class generates a call operator that invokes the greater-than operator of the underlying element type. For example, if `svec` is a `vector<string>`,

```
// passes a temporary function object that applies the < operator to two strings
sort(svec.begin(), svec.end(), greater<string>());
```

sorts the `vector` in descending order. The third argument is an unnamed object of type `greater<string>`. When `sort` compares elements, rather than applying the `<` operator for the element type, it will call the given `greater` function object. That object applies `>` to the `string` elements.

One important aspect of these library function objects is that the library guarantees that they will work for pointers. Recall that comparing two unrelated pointers is undefined (§ 3.5.3, p. 120). However, we might want to `sort` a `vector` of pointers based on their addresses in memory. Although it would be undefined for us to do so directly, we can do so through one of the library function objects:

```
vector<string *> nameTable;  // vector of pointers
// error: the pointers in nameTable are unrelated, so < is undefined
sort(nameTable.begin(), nameTable.end(),
     [](string *a, string *b) { return a < b; });
// ok: library guarantees that less on pointer types is well defined
sort(nameTable.begin(), nameTable.end(), less<string*>());
```

It is also worth noting that the associative containers use `less<key_type>` to order their elements. As a result, we can define a `set` of pointers or use a pointer as the key in a `map` without specifying `less` directly.

Exercises Section 14.8.2

Exercise 14.42: Using library function objects and adaptors, define an expression to

(a) Count the number of values that are greater than 1024
(b) Find the first string that is not equal to `pooh`
(c) Multiply all values by 2

Exercise 14.43: Using library function objects, determine whether a given `int` value is divisible by any element in a container of `int`s.

14.8.3 Callable Objects and `function`

C++ has several kinds of callable objects: functions and pointers to functions, lambdas (§ 10.3.2, p. 388), objects created by `bind` (§ 10.3.4, p. 397), and classes that overload the function-call operator.

Like any other object, a callable object has a type. For example, each lambda has its own unique (unnamed) class type. Function and function-pointer types vary by their return type and argument types, and so on.

However, two callable objects with different types may share the same **call signature**. The call signature specifies the type returned by a call to the object and the argument type(s) that must be passed in the call. A call signature corresponds to a function type. For example:

```
int(int, int)
```

is a function type that takes two `int`s and returns an `int`.

Different Types Can Have the Same Call Signature

Sometimes we want to treat several callable objects that share a call signature as if they had the same type. For example, consider the following different types of callable objects:

```
// ordinary function
int add(int i, int j) { return i + j; }
// lambda, which generates an unnamed function-object class
auto mod = [](int i, int j) { return i % j; };
// function-object class
struct divide {
    int operator()(int denominator, int divisor) {
        return denominator / divisor;
    }
};
```

Each of these callables applies an arithmetic operation to its parameters. Even though each has a distinct type, they all share the same call signature:

```
int(int, int)
```

We might want to use these callables to build a simple desk calculator. To do so, we'd want to define a **function table** to store "pointers" to these callables. When the program needs to execute a particular operation, it will look in the table to find which function to call.

In C++, function tables are easy to implement using a `map`. In this case, we'll use a `string` corresponding to an operator symbol as the key; the value will be the function that implements that operator. When we want to evaluate a given operator, we'll index the `map` with that operator and call the resulting element.

If all our functions were freestanding functions, and assuming we were handling only binary operators for type `int`, we could define the `map` as

```
// maps an operator to a pointer to a function taking two ints and returning an int
map<string, int(*)(int,int)> binops;
```

We could put a pointer to `add` into `binops` as follows:

```
// ok: add is a pointer to function of the appropriate type
binops.insert({"+", add}); // {"+", add} is a pair §11.2.3 (p. 426)
```

However, we can't store `mod` or an object of type `divide` in `binops`:

```
binops.insert({"%", mod}); // error: mod is not a pointer to function
```

The problem is that `mod` is a lambda, and each lambda has its own class type. That type does not match the type of the values stored in `binops`.

The Library `function` Type

We can solve this problem using a new library type named **`function`** that is defined in the `functional` header; Table 14.3 (p. 579) lists the operations defined by `function`.

C++ 11

`function` is a template. As with other templates we've used, we must specify additional information when we create a `function` type. In this case, that information is the call signature of the objects that this particular `function` type can represent. As with other templates, we specify the type inside angle brackets:

```
function<int(int, int)>
```

Here we've declared a `function` type that can represent callable objects that return an `int` result and have two `int` parameters. We can use that type to represent any of our desk calculator types:

```
function<int(int, int)> f1 = add;       // function pointer
function<int(int, int)> f2 = divide(); // object of a function-object class
function<int(int, int)> f3 = [](int i, int j) // lambda
                             { return i * j; };
cout << f1(4,2) << endl; // prints 6
cout << f2(4,2) << endl; // prints 2
cout << f3(4,2) << endl; // prints 8
```

We can now redefine our `map` using this `function` type:

```
// table of callable objects corresponding to each binary operator
// all the callables must take two ints and return an int
// an element can be a function pointer, function object, or lambda
map<string, function<int(int, int)>> binops;
```

We can add each of our callable objects, be they function pointers, lambdas, or function objects, to this `map`:

```
map<string, function<int(int, int)>> binops = {
    {"+", add},                      // function pointer
    {"-", std::minus<int>()},        // library function object
    {"/",  divide()},                // user-defined function object
    {"*", [](int i, int j) { return i * j; }}, // unnamed lambda
    {"%", mod} };                    // named lambda object
```

Our `map` has five elements. Although the underlying callable objects all have different types from one another, we can store each of these distinct types in the common `function<int(int, int)>` type.

As usual, when we index a `map`, we get a reference to the associated value. When we index `binops`, we get a reference to an object of type `function`. The `function` type overloads the call operator. That call operator takes its own arguments and passes them along to its stored callable object:

```
binops["+"](10, 5); // calls add(10, 5)
binops["-"](10, 5); // uses the call operator of the minus<int> object
binops["/"](10, 5); // uses the call operator of the divide object
binops["*"](10, 5); // calls the lambda function object
binops["%"](10, 5); // calls the lambda function object
```

Here we call each of the operations stored in `binops`. In the first call, the element we get back holds a function pointer that points to our `add` function. Calling `binops["+"](10, 5)` uses that pointer to call `add`, passing it the values `10` and `5`. In the next call, `binops["-"]`, returns a `function` that stores an object of type `std::minus<int>`. We call that object's call operator, and so on.

Overloaded Functions and `function`

We cannot (directly) store the name of an overloaded function in an object of type `function`:

```
int add(int i, int j) { return i + j; }
Sales_data add(const Sales_data&, const Sales_data&);
map<string, function<int(int, int)>> binops;
binops.insert( {"+", add} ); // error: which add?
```

One way to resolve the ambiguity is to store a function pointer (§ 6.7, p. 247) instead of the name of the function:

```
int (*fp)(int,int) = add; // pointer to the version of add that takes two ints
binops.insert( {"+", fp} ); // ok: fp points to the right version of add
```

Table 14.3: Operations on `function`

`function<T> f;`	`f` is a null `function` object that can store callable objects with a call signature that is equivalent to the function type `T` (i.e., `T` is *retType*`(`*args*`)`).
`function<T> f(nullptr);`	Explicitly construct a null `function`.
`function<T> f(obj);`	Stores a copy of the callable object `obj` in `f`.
`f`	Use `f` as a condition; `true` if `f` holds a callable object; `false` otherwise.
`f(`*args*`)`	Calls the object in `f` passing *args*.
Types defined as members of `function<T>`	
`result_type`	The type returned by this `function` type's callable object.

`argument_type`
`first_argument_type`
`second_argument_type`
	Types defined when `T` has exactly one or two arguments. If `T` has one argument, `argument_type` is a synonym for that type. If `T` has two arguments, `first_argument_type` and `second_argument_type` are synonyms for those argument types.

Alternatively, we can use a lambda to disambiguate:

```
// ok: use a lambda to disambiguate which version of add we want to use
binops.insert( {"+", [](int a, int b) {return add(a, b);} } );
```

The call inside the lambda body passes two `int`s. That call can match only the version of `add` that takes two `int`s, and so that is the function that is called when the lambda is executed.

The `function` class in the new library is not related to classes named `unary_function` and `binary_function` that were part of earlier versions of the library. These classes have been deprecated by the more general `bind` function (§ 10.3.4, p. 401).

Exercises Section 14.8.3

Exercise 14.44: Write your own version of a simple desk calculator that can handle binary operations.

14.9 Overloading, Conversions, and Operators

In § 7.5.4 (p. 294) we saw that a nonexplicit constructor that can be called with one argument defines an implicit conversion. Such constructors convert an object from the argument's type *to* the class type. We can also define conversions *from* the class type. We define a conversion from a class type by defining a conversion

operator. Converting constructors and conversion operators define **class-type conversions**. Such conversions are also referred to as **user-defined conversions**.

14.9.1 Conversion Operators

A **conversion operator** is a special kind of member function that converts a value of a class type to a value of some other type. A conversion function typically has the general form

```
operator type() const;
```

where *type* represents a type. Conversion operators can be defined for any type (other than `void`) that can be a function return type (§ 6.1, p. 204). Conversions to an array or a function type are not permitted. Conversions to pointer types—both data and function pointers—and to reference types are allowed.

Conversion operators have no explicitly stated return type and no parameters, and they must be defined as member functions. Conversion operations ordinarily should not change the object they are converting. As a result, conversion operators usually should be defined as `const` members.

A conversion function must be a member function, may not specify a return type, and must have an empty parameter list. The function usually should be `const`.

Defining a Class with a Conversion Operator

As an example, we'll define a small class that represents an integer in the range of 0 to 255:

```
class SmallInt {
public:
    SmallInt(int i = 0): val(i)
    {
        if (i < 0 || i > 255)
            throw std::out_of_range("Bad SmallInt value");
    }
    operator int() const { return val; }
private:
    std::size_t val;
};
```

Our `SmallInt` class defines conversions *to* and *from* its type. The constructor converts values of arithmetic type to a `SmallInt`. The conversion operator converts `SmallInt` objects to `int`:

```
SmallInt si;
si = 4; // implicitly converts 4 to SmallInt then calls SmallInt::operator=
si + 3; // implicitly converts si to int followed by integer addition
```

Although the compiler will apply only one user-defined conversion at a time (§ 4.11.2, p. 162), an implicit user-defined conversion can be preceded or followed by a standard (built-in) conversion (§ 4.11.1, p. 159). As a result, we can pass any arithmetic type to the `SmallInt` constructor. Similarly, we can use the converion operator to convert a `SmallInt` to an `int` and then convert the resulting `int` value to another arithmetic type:

```
// the double argument is converted to int using the built-in conversion
SmallInt si = 3.14; // calls the SmallInt(int) constructor
// the SmallInt conversion operator converts si to int;
si + 3.14; // that int is converted to double using the built-in conversion
```

Because conversion operators are implicitly applied, there is no way to pass arguments to these functions. Hence, conversion operators may not be defined to take parameters. Although a conversion function does not specify a return type, each conversion function must return a value of its corresponding type:

```
class SmallInt;
operator int(SmallInt&);                    // error: nonmember
class SmallInt {
public:
    int operator int() const;               // error: return type
    operator int(int = 0) const;            // error: parameter list
    operator int*() const { return 42; } // error: 42 is not a pointer
};
```

CAUTION: AVOID OVERUSE OF CONVERSION FUNCTIONS

As with using overloaded operators, judicious use of conversion operators can greatly simplify the job of a class designer and make using a class easier. However, some conversions can be misleading. Conversion operators are misleading when there is no obvious single mapping between the class type and the conversion type.

For example, consider a class that represents a `Date`. We might think it would be a good idea to provide a conversion from `Date` to `int`. However, what value should the conversion function return? The function might return a decimal representation of the year, month, and day. For example, July 30, 1989 might be represented as the `int` value 19800730. Alternatively, the conversion operator might return an `int` representing the number of days that have elapsed since some epoch point, such as January 1, 1970. Both these conversions have the desirable property that later dates correspond to larger integers, and so either might be useful.

The problem is that there is no single one-to-one mapping between an object of type `Date` and a value of type `int`. In such cases, it is better not to define the conversion operator. Instead, the class ought to define one or more ordinary members to extract the information in these various forms.

Conversion Operators Can Yield Suprising Results

In practice, classes rarely provide conversion operators. Too often users are more likely to be surprised if a conversion happens automatically than to be helped by

the existence of the conversion. However, there is one important exception to this rule of thumb: It is not uncommon for classes to define conversions to `bool`.

Under earlier versions of the standard, classes that wanted to define a conversion to `bool` faced a problem: Because `bool` is an arithmetic type, a class-type object that is converted to `bool` can be used in any context where an arithmetic type is expected. Such conversions can happen in surprising ways. In particular, if `istream` had a conversion to `bool`, the following code would compile:

```
int i = 42;
cin << i; // this code would be legal if the conversion to bool were not explicit!
```

This program attempts to use the output operator on an input stream. There is no `<<` defined for `istream`, so the code is almost surely in error. However, this code could use the `bool` conversion operator to convert `cin` to `bool`. The resulting `bool` value would then be promoted to `int` and used as the left-hand operand to the built-in version of the left-shift operator. The promoted `bool` value (either 1 or 0) would be shifted left 42 positions.

`explicit` Conversion Operators

C++ 11

To prevent such problems, the new standard introduced **`explicit` conversion operators**:

```
class SmallInt {
public:
    // the compiler won't automatically apply this conversion
    explicit operator int() const { return val; }
    // other members as before
};
```

As with an `explicit` constructor (§ 7.5.4, p. 296), the compiler won't (generally) use an `explicit` conversion operator for implicit conversions:

```
SmallInt si = 3;  // ok: the SmallInt constructor is not explicit
si + 3; // error: implicit is conversion required, but operator int is explicit
static_cast<int>(si) + 3; // ok: explicitly request the conversion
```

If the conversion operator is `explicit`, we can still do the conversion. However, with one exception, we must do so explicitly through a cast.

The exception is that the compiler will apply an `explicit` conversion to an expression used as a condition. That is, an `explicit` conversion will be used implicitly to convert an expression used as

- The condition of an `if`, `while`, or `do` statement
- The condition expression in a `for` statement header
- An operand to the logical NOT (`!`), OR (`||`), or AND (`&&`) operators
- The condition expression in a conditional (`?:`) operator

Conversion to `bool`

In earlier versions of the library, the IO types defined a conversion to `void*`. They did so to avoid the kinds of problems illustrated above. Under the new standard, the IO library instead defines an `explicit` conversion to `bool`.

Whenever we use a stream object in a condition, we use the `operator bool` that is defined for the IO types. For example,

```
while (std::cin >> value)
```

The condition in the `while` executes the input operator, which reads into `value` and returns `cin`. To evaluate the condition, `cin` is implicitly converted by the `istream operator bool` conversion function. That function returns `true` if the condition state of `cin` is `good` (§ 8.1.2, p. 312), and `false` otherwise.

Conversion to `bool` is usually intended for use in conditions. As a result, `operator bool` ordinarily should be defined as `explicit`.

Exercises Section 14.9.1

Exercise 14.45: Write conversion operators to convert a `Sales_data` to `string` and to `double`. What values do you think these operators should return?

Exercise 14.46: Explain whether defining these `Sales_data` conversion operators is a good idea and whether they should be `explicit`.

Exercise 14.47: Explain the difference between these two conversion operators:

```
struct Integral {
    operator const int();
    operator int() const;
};
```

Exercise 14.48: Determine whether the class you used in exercise 7.40 from § 7.5.1 (p. 291) should have a conversion to `bool`. If so, explain why, and explain whether the operator should be `explicit`. If not, explain why not.

Exercise 14.49: Regardless of whether it is a good idea to do so, define a conversion to `bool` for the class from the previous exercise.

14.9.2 Avoiding Ambiguous Conversions

If a class has one or more conversions, it is important to ensure that there is only one way to convert from the class type to the target type. If there is more than one way to perform a conversion, it will be hard to write unambiguous code.

There are two ways that multiple conversion paths can occur. The first happens when two classes provide mutual conversions. For example, mutual conversions exist when a class `A` defines a converting constructor that takes an object of class `B` and `B` itself defines a conversion operator to type `A`.

The second way to generate multiple conversion paths is to define multiple conversions from or to types that are themselves related by conversions. The most obvious instance is the built-in arithmetic types. A given class ordinarily ought to define at most one conversion to or from an arithmetic type.

Ordinarily, it is a bad idea to define classes with mutual conversions or to define conversions to or from two arithmetic types.

Argument Matching and Mutual Conversions

In the following example, we've defined two ways to obtain an `A` from a `B`: either by using `B`'s conversion operator or by using the `A` constructor that takes a `B`:

```
// usually a bad idea to have mutual conversions between two class types
struct B;
struct A {
    A() = default;
    A(const B&);          // converts a B to an A
    // other members
};
struct B {
    operator A() const; // also converts a B to an A
    // other members
};
A f(const A&);
B b;
A a = f(b); // error ambiguous: f(B::operator A())
            //              or f(A::A(const B&))
```

Because there are two ways to obtain an `A` from a `B`, the compiler doesn't know which conversion to run; the call to `f` is ambiguous. This call can use the `A` constructor that takes a `B`, or it can use the `B` conversion operator that converts a `B` to an `A`. Because these two functions are equally good, the call is in error.

If we want to make this call, we have to explicitly call the conversion operator or the constructor:

```
A a1 = f(b.operator A()); // ok: use B's conversion operator
A a2 = f(A(b));           // ok: use A's constructor
```

Note that we can't resolve the ambiguity by using a cast—the cast itself would have the same ambiguity.

Ambiguities and Multiple Conversions to Built-in Types

Ambiguities also occur when a class defines multiple conversions to (or from) types that are themselves related by conversions. The easiest case to illustrate—and one that is particularly problematic—is when a class defines constructors from or conversions to more than one arithmetic type.

For example, the following class has converting constructors from two different arithmetic types, and conversion operators to two different arithmetic types:

```
struct A {
    A(int = 0);     // usually a bad idea to have two
    A(double);      // conversions from arithmetic types
    operator int() const;      // usually a bad idea to have two
    operator double() const; // conversions to arithmetic types
    // other members
};
void f2(long double);
A a;
f2(a); // error ambiguous: f(A::operator int())
       //              or f(A::operator double())
long lg;
A a2(lg); // error ambiguous: A::A(int) or A::A(double)
```

In the call to `f2`, neither conversion is an exact match to `long double`. However, either conversion can be used, followed by a standard conversion to get to `long double`. Hence, neither conversion is better than the other; the call is ambiguous.

We encounter the same problem when we try to initialize `a2` from a `long`. Neither constructor is an exact match for `long`. Each would require that the argument be converted before using the constructor:

- Standard `long` to `double` conversion followed by `A(double)`
- Standard `long` to `int` conversion followed by `A(int)`

These conversion sequences are indistinguishable, so the call is ambiguous.

The call to `f2`, and the initialization of `a2`, are ambiguous because the standard conversions that were needed had the same rank (§ 6.6.1, p. 245). When a user-defined conversion is used, the rank of the standard conversion, if any, is used to select the best match:

```
short s = 42;
// promoting short to int is better than converting short to double
A a3(s);  // uses A::A(int)
```

In this case, promoting a `short` to an `int` is preferred to converting the `short` to a `double`. Hence `a3` is constructed using the `A::A(int)` constructor, which is run on the (promoted) value of `s`.

When two user-defined conversions are used, the rank of the standard conversion, if any, *preceding* or *following* the conversion function is used to select the best match.

Overloaded Functions and Converting Constructors

Choosing among multiple conversions is further complicated when we call an overloaded function. If two or more conversions provide a viable match, then the conversions are considered equally good.

As one example, ambiguity problems can arise when overloaded functions take parameters that differ by class types that define the same converting constructors:

CAUTION: CONVERSIONS AND OPERATORS

Correctly designing the overloaded operators, conversion constructors, and conversion functions for a class requires some care. In particular, ambiguities are easy to generate if a class defines both conversion operators and overloaded operators. A few rules of thumb can be helpful:

- **Don't define mutually converting classes—if class `Foo` has a constructor that takes an object of class `Bar`, do not give `Bar` a conversion operator to type `Foo`.**
- **Avoid conversions to the built-in arithmetic types. In particular, if you do define a conversion to an arithmetic type, then**
  - **Do not define overloaded versions of the operators that take arithmetic types. If users need to use these operators, the conversion operation will convert objects of your type, and then the built-in operators can be used.**
  - **Do not define a conversion to more than one arithmetic type. Let the standard conversions provide conversions to the other arithmetic types.**

The easiest rule of all: With the exception of an `explicit` conversion to `bool`, avoid defining conversion functions and limit non`explicit` constructors to those that are "obviously right."

```
struct C {
    C(int);
    //  other members
};
struct D {
    D(int);
    //  other members
};
void manip(const C&);
void manip(const D&);
manip(10); //  error ambiguous: manip(C(10)) or manip(D(10))
```

Here both `C` and `D` have constructors that take an `int`. Either constructor can be used to match a version of `manip`. Hence, the call is ambiguous: It could mean convert the `int` to `C` and call the first version of `manip`, or it could mean convert the `int` to `D` and call the second version.

The caller can disambiguate by explicitly constructing the correct type:

```
manip(C(10)); //  ok: calls manip(const C&)
```

Needing to use a constructor or a cast to convert an argument in a call to an overloaded function frequently is a sign of bad design.

Overloaded Functions and User-Defined Conversion

In a call to an overloaded function, if two (or more) user-defined conversions provide a viable match, the conversions are considered equally good. The rank of

any standard conversions that might or might not be required is not considered. Whether a built-in conversion is also needed is considered only if the overload set can be matched *using the same conversion function*.

For example, our call to `manip` would be ambiguous even if one of the classes defined a constructor that required a standard conversion for the argument:

```
struct E {
    E(double);
    // other members
};
void manip2(const C&);
void manip2(const E&);
// error ambiguous: two different user-defined conversions could be used
manip2(10); // manip2(C(10) or manip2(E(double(10)))
```

In this case, `C` has a conversion from `int` and `E` has a conversion from `double`. For the call `manip2(10)`, both `manip2` functions are viable:

- `manip2(const C&)` is viable because `C` has a converting constructor that takes an `int`. That constructor is an exact match for the argument.
- `manip2(const E&)` is viable because `E` has a converting constructor that takes a `double` and we can use a standard conversion to convert the `int` argument in order to use that converting constructor.

Because calls to the overloaded functions require *different* user-defined conversions from one another, this call is ambiguous. In particular, even though one of the calls requires a standard conversion and the other is an exact match, the compiler will still flag this call as an error.

In a call to an overloaded function, the rank of an additional standard conversion (if any) matters only if the viable functions require the same user-defined conversion. If different user-defined conversions are needed, then the call is ambiguous.

14.9.3 Function Matching and Overloaded Operators

Overloaded operators are overloaded functions. Normal function matching (§ 6.4, p. 233) is used to determine which operator—built-in or overloaded—to apply to a given expression. However, when an operator function is used in an expression, the set of candidate functions is broader than when we call a function using the call operator. If `a` has a class type, the expression `a` *sym* `b` might be

```
a.operatorsym(b); // a has operatorsym as a member function
operatorsym(a, b);// operatorsym is an ordinary function
```

Unlike ordinary function calls, we cannot use the form of the call to distinquish whether we're calling a nonmember or a member function.

EXERCISES SECTION 14.9.2

Exercise 14.50: Show the possible class-type conversion sequences for the initializations of `ex1` and `ex2`. Explain whether the initializations are legal or not.

```
struct LongDouble {
    LongDouble(double = 0.0);
    operator double();
    operator float();
};
LongDouble ldObj;
int ex1 = ldObj;
float ex2 = ldObj;
```

Exercise 14.51: Show the conversion sequences (if any) needed to call each version of `calc` and explain why the best viable function is selected.

```
void calc(int);
void calc(LongDouble);
double dval;
calc(dval); //  which calc?
```

When we use an overloaded operator with an operand of class type, the candidate functions include ordinary nonmember versions of that operator, as well as the built-in versions of the operator. Moreover, if the left-hand operand has class type, the overloaded versions of the operator, if any, defined by that class are also included.

When we call a named function, member and nonmember functions with the same name do *not* overload one another. There is no overloading because the syntax we use to call a named function distinguishes between member and nonmember functions. When a call is through an object of a class type (or through a reference or pointer to such an object), then only the member functions of that class are considered. When we use an overloaded operator in an expression, there is nothing to indicate whether we're using a member or nonmember function. Therefore, both member and nonmember versions must be considered.

The set of candidate functions for an operator used in an expression can contain both nonmember and member functions.

As an example, we'll define an addition operator for our `SmallInt` class:

```
class SmallInt {
    friend
    SmallInt operator+(const SmallInt&, const SmallInt&);
public:
    SmallInt(int = 0);                    //  conversion from int
    operator int() const { return val; } //  conversion to int
private:
    std::size_t val;
};
```

We can use this class to add two `SmallInt`s, but we will run into ambiguity problems if we attempt to perform mixed-mode arithmetic:

```
SmallInt s1, s2;
SmallInt s3 = s1 + s2;   // uses overloaded operator+
int i = s3 + 0;          // error: ambiguous
```

The first addition uses the overloaded version of `+` that takes two `SmallInt` values. The second addition is ambiguous, because we can convert `0` to a `SmallInt` and use the `SmallInt` version of `+`, or convert `s3` to `int` and use the built-in addition operator on `int`s.

Providing both conversion functions to an arithmetic type and overloaded operators for the same class type may lead to ambiguities between the overloaded operators and the built-in operators.

EXERCISES SECTION 14.9.3

Exercise 14.52: Which `operator+`, if any, is selected for each of the addition expressions? List the candidate functions, the viable functions, and the type conversions on the arguments for each viable function:

```
struct LongDouble {
    // member operator+ for illustration purposes; + is usually a nonmember
    LongDouble operator+(const SmallInt&);
    // other members as in § 14.9.2 (p. 587)
};
LongDouble operator+(LongDouble&, double);
SmallInt si;
LongDouble ld;
ld = si + ld;
ld = ld + si;
```

Exercise 14.53: Given the definition of `SmallInt` on page 588, determine whether the following addition expression is legal. If so, what addition operator is used? If not, how might you change the code to make it legal?

```
SmallInt s1;
double d = s1 + 3.14;
```

Chapter Summary

An overloaded operator must either be a member of a class or have at least one operand of class type. Overloaded operators have the same number of operands, associativity, and precedence as the corresponding operator when applied to the built-in types. When an operator is defined as a member, its implicit `this` pointer is bound to the first operand. The assignment, subscript, function-call, and arrow operators must be class members.

Objects of classes that overload the function-call operator, `operator()`, are known as "function objects." Such objects are often used in combination with the standard algorithms. Lambda expressions are succinct ways to define simple function-object classes.

A class can define conversions to or from its type that are used automatically. Non`explicit` constructors that can be called with a single argument define conversions from the parameter type to the class type; non`explicit` conversion operators define conversions from the class type to other types.

Defined Terms

call signature Represents the interface of a callable object. A call signature includes the return type and a comma-separated list of argument types enclosed in parentheses.

class-type conversion Conversions to or from class types are defined by constructors and conversion operators, respectively. Non`explicit` constructors that take a single argument define a conversion from the argument type to the class type. Conversion operators define conversions from the class type to the specified type.

conversion operator A member function that defines a conversions from the class type to another type. A conversion operator must be a member of the class from which it converts and is usually a `const` member. These operators have no return type and take no parameters. They return a value convertible to the type of the conversion operator. That is, `operator int` returns an `int`, `operator string` returns a `string`, and so on.

explicit conversion operator Conversion operator preceeded by the `explicit` keyword. Such operators are used for implicit conversions only in conditions.

function object Object of a class that defines an overloaded call operator. Function objects can be used where functions are normally expected.

function table Container, often a `map` or a `vector`, that holds values that can be called.

function template Library template that can represent any callable type.

overloaded operator Function that redefines the meaning of one of the built-in operators. Overloaded operator functions have the name `operator` followed by the symbol being defined. Overloaded operators must have at least one operand of class type. Overloaded operators have the same precedence, associativity and number of operands as their built-in counterparts.

user-defined conversion A synonym for class-type conversion.